

When Should Reinforcement Learning Algorithms Switch? A Study of Phase-Based Training in Continuous Control

Stanford CS224R

Repo: <https://github.com/dcunhrya/RL-translational-dynamics>

Ryan D'Cunha Ethan Hersch Abhinav Chinta
Department of Computer Science, Stanford University
{rdcunha, ehersch, achinta}@stanford.edu

1 Abstract

Deep reinforcement learning (RL) algorithms are typically evaluated as monolithic training procedures, despite the fact that different algorithms may be better suited to different phases of learning. Early training may require a stable non-random initialization, middle training may benefit from sample-efficient improvement, and late training may require conservative refinement without destructive policy updates. This project studies whether RL algorithms should instead be treated as phase-specific components of a broader training schedule.

We propose a mechanistic investigation of algorithm sequencing in continuous-control RL. Rather than asking whether Proximal Policy Optimization (PPO) or Soft Actor-Critic (SAC) is universally superior, we ask whether switching between algorithms under a fixed environment-step budget can improve sample efficiency, stability, or final return. We evaluate single-algorithm PPO and SAC baselines against fixed and adaptive handoff schedules, including $\text{SAC} \rightarrow \text{PPO}$ and $\text{PPO} \rightarrow \text{SAC}$. We frame each handoff as a transfer problem across a phase boundary, where the transferred substrate may include policy behavior, value estimates, replay data, or offline initialization. Because SAC and PPO use incompatible actor architectures in our implementation, policy transfer is implemented through behavioral distillation rather than direct weight loading. Our central hypothesis is that RL training decomposes into phases: an early phase in which imitation learning, offline RL, or simplified-environment pretraining can provide a useful policy prior; a middle phase in which off-policy methods such as SAC can efficiently improve through replay-based learning; and a late phase in which conservative updates may stabilize performance. To test this hypothesis, we study behavioral policy distillation, value initialization, replay-buffer transfer, adaptive switching, and offline-assisted warm starts using behavior cloning (BC), Advantage-Weighted Actor-Critic (AWAC), and easy-environment SAC pretraining.

We evaluate these schedules in MuJoCo continuous-control environments, focusing on Hopper-v4 and Walker2d-v4. Performance is measured using final return, area under the learning curve (AUC), variance across seeds, collapse frequency, and qualitative inspection of training dynamics. Across completed 500k-step PPO/SAC experiments, SAC remains the strongest standalone online baseline, achieving the best AUC and final return on both Hopper-v4 and Walker2d-v4. $\text{SAC} \rightarrow \text{PPO}$ handoffs improve over pure PPO in AUC but fail to match continued SAC training, suggesting that switching away from a still-improving off-policy learner is costly. A Hopper-v4 long-horizon check further supports this interpretation, as SAC continues to improve beyond the 500k-step budget.

In the reverse direction, $\text{PPO} \rightarrow \text{SAC}$ schedules reveal a clearer phase-allocation effect. Early handoffs substantially outperform late handoffs, indicating that PPO can provide a useful warm start only when SAC receives sufficient post-switch budget for sample-efficient improvement. A simple adaptive no-improvement trigger switches near 150k steps in 500k-step runs and performs strongly, especially on Walker2d-v4, suggesting that adaptive switching can reduce sensitivity to manually chosen switch fractions. Value initialization affects handoff behavior, but its benefits are

environment-dependent and statistically inconclusive: source-aligned and self-warmup value transfer help in some settings but do not provide a universal improvement.

Offline-assisted sequencing changes the picture without overturning the phase-allocation thesis. In Experiment 4, $BC \rightarrow SAC$ achieves strong final returns on Hopper-v4 and Walker2d-v4, while $BC \rightarrow PPO$ remains weak, showing that the same offline policy prior interacts very differently with the online target algorithm. $AWAC \rightarrow SAC$ is also viable but does not clearly outperform $BC \rightarrow SAC$ in the completed slice. The three-phase $BC \rightarrow SAC \rightarrow PPO$ schedule underperforms $BC \rightarrow SAC$ alone, echoing the online $SAC \rightarrow PPO$ failure mode and suggesting that a late PPO phase does not reliably stabilize a strong SAC trajectory. Interleaved BC anchoring also fails to improve over one-time BC initialization. Finally, easy-environment SAC pretraining alone collapses, but Easy SAC $\rightarrow$ real SAC reaches strong Hopper-v4 final returns, suggesting that curriculum-style pretraining is useful only when followed by real-environment fine-tuning.

These results suggest that algorithm sequencing is not a universal replacement for strong single-algorithm training. Instead, it provides a framework for understanding RL as phase allocation under a fixed interaction budget. The completed experiments show that switch direction, switch timing, target algorithm, and transfer mechanism substantially shape learning dynamics. Online-only and offline-assisted schedules should be interpreted as separate budget categories, but together they support the same conclusion: successful sequencing depends less on combining algorithms naively and more on matching each phase to the learner that can best exploit the transferred policy, value, or data.

2 Introduction

Deep reinforcement learning (RL) has produced strong empirical results across continuous-control, robotics, and simulated decision-making tasks, yet training remains sensitive to algorithm choice, initialization, and interaction budget. Standard empirical practice treats an RL algorithm as a fixed end-to-end training procedure: a policy is initialized, trained with a single update rule, and evaluated by its final return or sample efficiency. This framing is convenient for benchmarking, but it obscures the fact that RL training is not a homogeneous process. The demands placed on the learner early in training may differ substantially from those later in training. Early learning often requires broad exploration, non-random behavior, and useful state coverage; intermediate learning may require sample-efficient improvement from collected experience; and late learning may require stable updates that preserve already useful behavior.

This observation motivates a phase-based view of RL training. Rather than treating algorithms as monolithic choices, one can ask whether different algorithms are better suited to different stages of learning. On-policy methods such as Proximal Policy Optimization (PPO) [1] are often valued for stable policy updates, while off-policy methods such as Soft Actor-Critic (SAC) [2] can reuse replay data and often achieve strong sample efficiency in continuous-control domains. Similarly, offline or imitation-based methods can provide useful initial policies before online fine-tuning begins. These differences suggest that the relevant design question may not be which algorithm is universally best, but how learning should be allocated across algorithms, initialization, and transfer.

We study this question through algorithm sequencing which involves training schedules that divide a fixed interaction budget across multiple learning phases. The target learner may inherit policy behavior, value estimates, replay data, or a pretrained policy prior. Whether this inheritance is useful depends on both the source of the transferred information and the update dynamics of the target algorithm. A policy that is beneficial for one learner may be poorly preserved by another. A value estimate calibrated under one objective may be mismatched to a different target update.

This paper investigates algorithm sequencing in MuJoCo continuous-control environments [?], focusing on interactions among PPO, SAC, and offline-assisted warm starts. We evaluate sequencing as a mechanistic question rather than a simple leaderboard problem: when does a first phase provide useful initialization for a second phase, and when does switching disrupt a productive learner? We therefore consider both online handoffs, such as PPO $\rightarrow$ SAC and SAC $\rightarrow$ PPO, and offline-assisted schedules using behavior cloning (BC) and Advantage-Weighted Actor-Critic (AWAC) [3] from D4RL expert data [4]. These settings allow us to test whether sequencing effects arise from algorithm direction, switch timing, value initialization, or the interaction between an offline policy prior and the online target learner.

Our study supports a cautious view of algorithm sequencing. Strong single algorithms, especially SAC, remain difficult to outperform under matched online budgets. However, handoff experiments reveal structure that is hidden by single-algorithm comparisons: switch direction and timing substantially affect learning, value transfer is environment-dependent, and offline policy priors interact strongly with the target online algorithm. In particular, warm-starting SAC from offline or imitation policies can be effective, whereas switching from a strong SAC trajectory into PPO often fails to preserve performance. These results suggest that sequencing should not be understood as a universal performance trick, but as a framework for reasoning about phase allocation, transfer compatibility, and adaptive training schedules in deep RL.

The contributions of this paper are:

1. We frame deep RL training as a phase-allocation problem rather than a fixed single-algorithm procedure.
2. We formulate algorithm handoffs as transfer problems involving policy behavior, value estimates, and data.
3. We empirically study PPO/SAC sequencing under matched online interaction budgets in MuJoCo continuous-control tasks.
4. We evaluate offline-assisted warm starts and show that their effectiveness depends strongly on the target online learner.

3 Related Work

Online deep reinforcement learning. Deep RL algorithms are commonly compared as fixed end-to-end procedures. PPO [1] is an on-policy policy-gradient method that uses a clipped surrogate objective to improve update stability, building on trust-region ideas from TRPO [5]. SAC [2] instead uses an off-policy maximum-entropy actor-critic objective, allowing replay-based learning and encouraging exploration through entropy regularization. These methods represent different points in the stability–sample-efficiency tradeoff, motivating our study of whether they should be used as phase-specific components rather than mutually exclusive training choices. PPO is proposed as a simpler trust-region-style policy optimization method, while SAC is explicitly framed as off-policy maximum-entropy RL. :contentReference[oaicite:0]index=0

Offline and imitation-based initialization. A separate line of work studies how prior data can initialize or accelerate RL. Behavior cloning treats imitation learning as supervised prediction of expert actions and has long been used to obtain non-random policies from demonstrations [6]. D4RL provides standardized offline RL datasets, including expert and mixed-quality data, for evaluating data-driven RL methods [4]. AWAC [3] and IQL [7] address offline-to-online or offline RL by learning from previously collected data while limiting failures from distribution shift. These works motivate our offline-assisted schedules, but our focus is not only whether offline pretraining helps; rather, we ask how the benefit of a policy prior depends on the online target algorithm. D4RL was introduced for benchmarking offline RL datasets, AWAC specifically targets offline pretraining followed by online fine-tuning, and IQL avoids querying values of unseen actions during offline policy improvement. :contentReference[oaicite:1]index=1

Algorithm sequencing and phase allocation. Prior work largely evaluates PPO, SAC, imitation learning, and offline RL as standalone methods or as pretraining followed by fine-tuning. Our work instead treats algorithm changes as explicit handoffs between phases. This framing makes the transfer substrate central: a handoff may transfer policy behavior, value estimates, replay data, or an offline policy prior. By comparing PPO → SAC, SAC → PPO, BC → SAC, BC → PPO, AWAC-assisted schedules, and curriculum-style easy-environment pretraining, we study when phase transitions help and when they disrupt an otherwise productive learner. This positions sequencing as a mechanism for analyzing compatibility between initialization, target algorithm, and remaining interaction budget, rather than as a simple replacement for strong single-algorithm baselines.

4 Approach

The core experimental question is given some fixed environment-step budget B , how policy training can be split into different phases or how transfer learning across tasks and algorithms be applied for RL. Across this budget, we can decompose the phases into k schedules $A_1, \dots, A_k$ where each received a portion of the budget and may transfer policy behavior, value estimates, replay data, or pretrained initialization onto the next.

Some schedules can be **online only**: they leverage data from current rollouts and leverage online algorithms such as PPO or SAC. Others can be **offline-assisted**: stages may involve pretraining with behavior cloning, AWAC, IQL and then transition to online methods.

4.1 Phase-Based Scheduler

There are a few phases of this schedule. The **warmup phase** involves producing a simple non-random policy. This can be done with PPO warmup, behavior cloning, AWAC, or using SAC on an easy environment to experiment with transfer learning. The **online improvement stage** leverages strong online learners like SAC to improve rewards quickly: algorithms with strong sample efficiency benefit here. The final phase we propose is the **refinement phase** which involves more conservative late-stage updates to reduce final variance and ensure a stable policy is achieved. This makes PPO a well-suited algorithm for our final refinement phase. However, it can also be used for a bulk of the work in the online improvement phase (though often not as sample-efficient as SAC).

4.2 Algorithms

A selection of algorithms is chosen for each stage of this process.

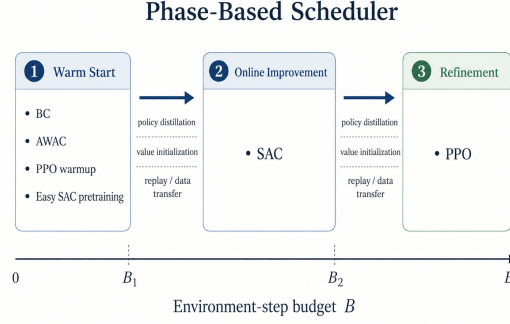


Figure 1: **Phase-based training scheduler.** We frame reinforcement learning under a fixed environment-step budget B as a sequence of phases. In the warm-start phase, methods such as BC, AWAC, PPO warmup, or Easy SAC pretraining provide a non-random initialization. The online improvement phase uses SAC for sample-efficient learning, and the refinement phase uses PPO as a final policy-improvement stage. Information can be transferred across phase boundaries through policy distillation, value initialization, and replay/data transfer.

Proximal Policy Optimization (PPO) [1] is a gradient-based method with a clipped objective

$$J(\theta) = \mathbb{E}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)], \text{ where } r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$$

This objective limits the new policy π_θ to stay close to the old policy $\pi_{\theta_{old}}$ so on-policy learning can be performed in a stable way and reuse sample batches of data for multiple gradient steps.

Soft Actor-Critic (SAC) [2] is an off-policy actor-critic algorithm that optimizes a maximum-entropy objective. Rather than only maximizing expected return, SAC also encourages the policy to maintain high entropy:

$$J(\pi) = \sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))],$$

where α controls the tradeoff between reward maximization and exploration. This entropy term encourages the agent to continue exploring actions instead of prematurely collapsing to a deterministic policy.

SAC is especially useful in the online improvement phase because it is off-policy and can reuse previously collected transitions through a replay buffer. Compared to PPO, which is typically more stable because of its clipped on-policy updates, SAC is often more sample efficient since each environment interaction can contribute to many gradient updates. In our scheduler, this makes SAC a natural choice for phases where the goal is to improve performance quickly under a fixed environment-step budget B . However, because SAC performs aggressive off-policy updates, we also investigate whether switching from SAC to a more conservative method such as PPO in the final refinement phase can reduce late-stage variance and produce a more stable final policy. This is the broad motivation for why SAC can be used during the online improvement stage prior to refinement.

Offline algorithms such as Implicit Q-Learning (IQL) [7] and Advantage-Weighted Actor-Critic (AWAC) [3] can be used when the agent has access to pre-collected trajectories before online interaction begins. These methods are useful in the warmup phase because they can initialize the policy using existing data without requiring additional environment steps. IQL avoids explicit out-of-distribution action maximization by learning value functions from the dataset and extracting a policy through advantage-weighted regression. AWAC similarly updates the policy toward actions in the dataset that have high estimated advantage, which makes it a natural bridge between offline pretraining and online fine-tuning.

We also consider behavior cloning (BC) as a simpler offline pretraining method. BC trains the policy directly to imitate actions from the dataset using a supervised learning objective,

$$J_{BC}(\theta) = \mathbb{E}_{(s,a) \sim \mathcal{D}} [-\log \pi_\theta(a | s)].$$

While BC does not explicitly optimize reward, it can provide a useful non-random initialization that allows subsequent online algorithms to begin from more structured behavior. In our scheduler, we evaluate both using BC as an initial warmup stage and switching between BC-style supervised updates and reinforcement learning updates during training. This lets us test whether imitation-based stabilization can improve early learning or reduce variance when combined with sample-efficient online methods such as SAC.

4.3 Handoff Mechanisms

The first dimension of experimentation in our work is how information is transferred across phase boundaries. A schedule is not only defined by the sequence of algorithms, but also by what the later phase inherits from the earlier phase. We therefore treat each handoff as a transfer operation rather than simply restarting training with a new optimizer.

At a switch point t_s , the source phase has trained a policy π_{src} and, depending on the algorithm, may also have learned value estimates, Q-functions, or a replay buffer. The target phase initializes a new learner π_{tgt} and may inherit one or more of these objects. We study three main transfer substrates: policy behavior, value estimates, and offline or pretrained initialization.

Policy transfer by behavioral distillation. PPO and SAC use different actor architectures in our implementation, so we do not transfer policies by directly copying neural network weights. Instead, we transfer behavior through distillation. Given a collection of states $\mathcal{S}$ sampled from source rollouts, replay data, or offline demonstrations, the target actor is trained to match the actions produced by the source policy:

$$\mathcal{L}_{\text{distill}}(\theta) = \mathbb{E}_{s \sim \mathcal{S}} \left[\|\mu_{\theta}(s) - \mu_{\text{src}}(s)\|_2^2 \right],$$

where $\mu_{\theta}(s)$ is the mean action of the target policy and $\mu_{\text{src}}(s)$ is the mean action of the source policy. This gives the target algorithm a behaviorally similar initialization while allowing it to retain its native architecture and update rule. We use this same distillation interface for online handoffs such as SAC $\rightarrow$ PPO and PPO $\rightarrow$ SAC, as well as offline-assisted warm starts such as BC $\rightarrow$ SAC and AWAC $\rightarrow$ SAC.

Value initialization. Policy transfer alone may be insufficient if the target algorithm starts with poorly calibrated value estimates. We therefore ablate how value functions are initialized at the handoff. In the online handoff experiments, we compare three settings:

$$\text{value-init} \in \{\text{random}, \text{self-warmup}, \text{source-aligned}\}.$$

In the **random** condition, the target value function or critic is initialized normally. In the **self-warmup** condition, the target value function is warmed up using its own native bootstrapping objective before full online training resumes. In the **source-aligned** condition, the target value function is trained to match value targets derived from the source phase. For SAC $\rightarrow$ PPO, this means initializing the PPO value function using value estimates induced by the SAC critic over replay states. For PPO $\rightarrow$ SAC, this means initializing SAC critics using return targets derived from PPO rollouts. Holding policy distillation fixed while varying value initialization allows us to test whether value transfer explains handoff performance.

Offline and pretrained policy initialization. For offline-assisted schedules, the source policy is trained before online interaction with the target environment begins. Behavior cloning learns from expert demonstrations, while AWAC uses offline RL updates on the same pre-collected data. These policies are then transferred into PPO or SAC using the same behavioral distillation mechanism described above. This lets us separate two questions: whether the offline method can produce a useful policy prior, and whether the target online learner can preserve and improve that prior.

We also evaluate a curriculum-style transfer setting in which SAC is first trained on an easier version of the environment and then transferred into SAC on the original benchmark. This tests whether a policy learned from a simplified task can serve as a warm start for the real task. Because this pretraining uses additional interaction in a modified environment, we report it separately from matched online-only comparisons.

Switch timing. A handoff is also defined by when it occurs. For fixed schedules, the switch step is determined by a fraction f of the total online budget:

$$t_s = \lfloor fB \rfloor, \quad f \in \{0.25, 0.50, 0.75\}.$$

This fixed-fraction sweep tests whether the first phase is useful only for early initialization or whether longer first-phase training improves the final policy. For adaptive schedules, the switch is triggered by training dynamics rather than a predetermined fraction. In our adaptive PPO $\rightarrow$ SAC experiments, PPO runs for a minimum first-phase budget and then switches to SAC when evaluation return fails to improve for a fixed patience window. This produces a simple reproducible scheduler:

$$t_s = \min \{t \geq t_{\min} : \text{NoImprove}(t, N)\},$$

where $t_{\min}$ is the minimum first-phase budget and N is the patience measured in evaluation checkpoints. In our main adaptive runs, $t_{\min} = 0.25B$ and $N = 3$.

Together, these handoff mechanisms allow us to distinguish several possible explanations for sequencing behavior. If policy distillation helps but value initialization does not, the benefit likely comes from behavior transfer rather than critic transfer. If offline policies help SAC but not PPO, then the same warm start is being used differently by different target algorithms. If early switches outperform late switches, then the main bottleneck is phase allocation: the second algorithm needs enough remaining budget to exploit the transferred initialization.

4.4 Online Handoff Schedules

We first study sequencing using only online interaction with the target MuJoCo environment. These experiments compare single-algorithm PPO and SAC baselines against two-algorithm handoff schedules under the same total environment-step budget B . The online-only setting is the cleanest test of phase allocation because all methods collect experience from the same target environment and are evaluated under matched interaction budgets.

The primary online handoff schedules are

$$\text{SAC} \rightarrow \text{PPO} \quad \text{and} \quad \text{PPO} \rightarrow \text{SAC}.$$

For each direction, training begins with the source algorithm until a switch step t_s , after which the target algorithm continues training for the remaining budget $B - t_s$. We evaluate fixed switch fractions

$$f \in \{0.25, 0.50, 0.75\}, \quad t_s = \lfloor fB \rfloor.$$

This sweep tests whether the handoff is useful only as a short warm start, whether a balanced split is best, or whether the first algorithm should receive most of the budget.

For SAC $\rightarrow$ PPO, the motivating hypothesis is that SAC can quickly learn a high-return policy through sample-efficient off-policy updates, while PPO may provide conservative late-stage refinement. If this hypothesis holds, PPO should preserve the strong SAC policy and reduce late-stage instability. However, this schedule can also fail if SAC is still improving at the switch point or if PPO cannot preserve the transferred behavior.

For PPO $\rightarrow$ SAC, the motivating hypothesis is different. PPO may provide a stable non-random initialization and useful early state distribution, after which SAC can use replay-based off-policy learning to improve more efficiently. This direction tests whether PPO is more useful as a warm-start phase than as a final refinement phase. We therefore pay particular attention to switch timing: if PPO consumes too much of the budget, SAC may not have enough remaining interaction to exploit the warm start.

In both directions, policy transfer is implemented through behavioral distillation rather than direct weight transfer. We also run value-initialization ablations to test whether critic or value transfer explains handoff performance. Holding policy transfer fixed, we compare target value initialization conditions:

$$\text{random}, \quad \text{self-warmup}, \quad \text{source-aligned}.$$

This allows us to distinguish whether a handoff succeeds because the target receives useful policy behavior, useful value estimates, or simply enough remaining budget to learn after the switch.

Finally, for PPO $\rightarrow$ SAC we evaluate a simple adaptive switch rule. Instead of selecting t_s manually, PPO trains for a minimum first phase and then switches to SAC when evaluation return fails to

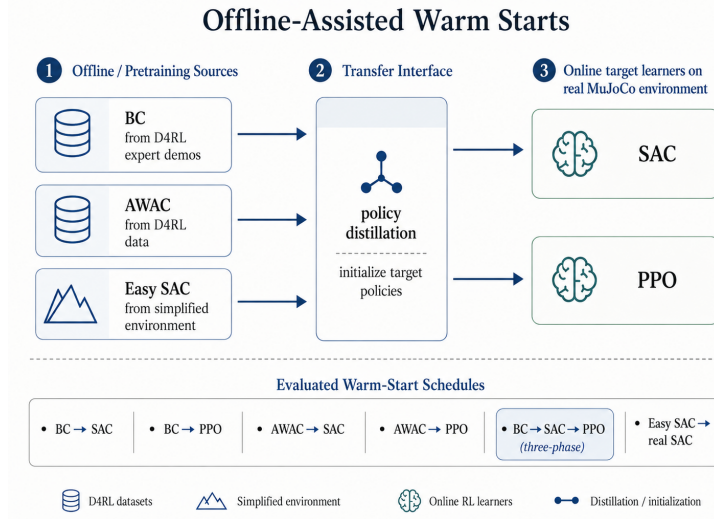


Figure 2: **Offline-assisted warm-start schedules.** We evaluate whether policies pretrained from offline demonstrations or simplified environments can improve downstream online training. Behavior cloning (BC) and AWAC are trained from D4RL data, while Easy SAC is pretrained in a simplified environment. These source policies are transferred through behavioral distillation into SAC or PPO target learners on the real MuJoCo task, enabling comparisons such as BC→SAC, BC→PPO, AWAC→SAC, AWAC→PPO, BC→SAC→PPO, and Easy SAC→real SAC.

improve for a fixed patience window. In our main adaptive runs, the minimum first phase is $0.25B$ and the patience is three evaluation checkpoints. This adaptive schedule tests whether a simple plateau detector can reduce sensitivity to fixed switch fractions while remaining explicit and reproducible.

4.5 Offline-Assisted Warm Starts

Experiment 4 extends the sequencing framework to offline-assisted warm starts. In these experiments, the first phase may use pre-collected demonstrations or offline RL before online fine-tuning begins. Because these methods use data or pretraining outside the target online interaction budget, we report offline dataset size and offline updates separately from online environment steps. These runs should therefore be interpreted as testing transfer compatibility and warm-start usefulness, not as strictly environment-step-equivalent comparisons against pure online PPO or SAC.

The first offline-assisted method is behavior cloning (BC). BC trains a policy on D4RL expert demonstrations using supervised imitation. The resulting policy is then transferred into PPO or SAC through the same behavioral distillation interface used in the online handoff experiments. We evaluate

$$\text{BC}, \quad \text{BC} \rightarrow \text{PPO}, \quad \text{BC} \rightarrow \text{SAC}.$$

This tests whether a policy-only offline prior is preserved and improved differently by on-policy and off-policy learners. Since BC does not learn a value function optimized for the online target algorithm, this experiment isolates the effect of policy initialization.

We also evaluate the full three-phase schedule

$$\text{BC} \rightarrow \text{SAC} \rightarrow \text{PPO}.$$

This schedule most directly reflects the proposed phase decomposition: imitation provides a warm start, SAC performs sample-efficient online improvement, and PPO attempts late-stage conservative refinement. The purpose of this experiment is not to assume that the three-phase scheduler will dominate, but to test whether the final PPO phase actually preserves or stabilizes the strong policy produced by BC and SAC.

To test whether a value-aware offline method changes the transfer behavior, Experiment 4 includes Advantage-Weighted Actor-Critic (AWAC). AWAC uses offline RL updates to train a policy from demonstration data while weighting actions according to estimated advantage. We evaluate

$$\text{AWAC}, \quad \text{AWAC} \rightarrow \text{PPO}, \quad \text{AWAC} \rightarrow \text{SAC}.$$

These schedules test whether an offline method that uses value information provides a more useful initialization than BC alone, and whether that initialization is better exploited by SAC or PPO.

Experiment 4 also includes interleaved BC anchoring during SAC. In these runs, SAC is initialized from the BC policy and periodically receives short BC-style anchoring updates every K environment steps:

BC-anchor SAC(K).

This tests whether repeated imitation regularization helps maintain the useful behavior learned from demonstrations, or whether ongoing anchoring constrains SAC and prevents improvement beyond the demonstrated policy.

Finally, we evaluate a curriculum-style transfer setting using an easier version of the environment. SAC is first trained on a simplified or more forgiving variant of the task, then transferred into SAC on the original benchmark:

Easy SAC $\rightarrow$ Real SAC.

This experiment asks whether a policy learned through RL in an easier environment can serve as a useful starter policy for the real environment. It differs from BC and AWAC because the warm start comes from interaction with a modified task rather than expert demonstrations. As with other offline-assisted or pretraining methods, we report this as a separate budget category.

Together, the Experiment 4 schedules test whether successful sequencing depends only on having a strong initial policy, or also on choosing a target algorithm capable of preserving and improving that policy. The comparison between BC $\rightarrow$ SAC and BC $\rightarrow$ PPO, and similarly between AWAC $\rightarrow$ SAC and AWAC $\rightarrow$ PPO, directly measures how the same warm start interacts with different online update rules.

4.6 Experimental Protocol

We evaluate all schedules on MuJoCo continuous-control environments, focusing on Hopper-v4 and Walker2d-v4. These environments are used for the main PPO/SAC handoff experiments, value-transfer ablations, adaptive switching runs, and offline-assisted warm-start schedules. Additional MuJoCo environments are considered as broader benchmarks where available, but the completed core analysis centers on Hopper-v4 and Walker2d-v4.

Unless otherwise stated, online training uses a budget of 500k environment steps. Selected long-horizon checks are run for 1M environment steps to test whether observed failures are caused by poor sequencing or simply by insufficient training time. Core online and offline-transfer arms are run with five random seeds when feasible. Stretch experiments, adaptive schedules, interleaved-BC sweeps, easy-environment transfer, and long-horizon checks are typically run with three seeds. Standalone offline pretraining evaluations, such as BC-only or AWAC-only policy evaluation, are sometimes reported with one seed and are used mainly to verify that the warm-start policy is non-random.

All experiments log evaluation return periodically during training, along with environment steps, gradient updates, wall-clock time, switch step, switch reason, phase identifier, and relevant hyperparameters. Handoff runs explicitly record whether the current phase is pre-switch training, distillation, value warmup, online fine-tuning, BC anchoring, or adaptive switching. Runs are launched through Modal and tracked with Weights & Biases where available. Experiment names encode the environment, seed, algorithm sequence, switch rule, budget, and run identifier. Configurations and outputs are saved separately for each run, and old experiment outputs are not overwritten.

4.7 Metrics

We report several metrics to capture more than final performance. The primary endpoint metric is **final return**, defined as the mean evaluation return at the end of the online training budget. To measure sample efficiency over the full learning process, we compute **normalized area under the learning curve (AUC)** from evaluation return versus environment steps. Higher AUC indicates that a method learns useful behavior earlier, even if final returns are similar.

To evaluate robustness, we report variance across seeds, **worst-seed return**, and **collapse count**. Worst-seed return captures the lower tail of performance, while collapse count records how often a run falls below a low-return failure threshold. These metrics are important because a schedule that

occasionally produces strong policies but frequently collapses is less useful than one with slightly lower mean return but more stable behavior.

For handoff experiments, we also measure the **handoff transient**, defined as the change in evaluation return immediately before and after the switch. This helps identify whether the target algorithm preserves or disrupts the source policy. Where available, we measure **policy retention** using action-space distance or KL-style diagnostics between the transferred source policy and the target policy over time. For value-transfer experiments, we log value warmup losses and related value-quality diagnostics. Together, these metrics let us distinguish final performance from learning speed, stability, and transfer compatibility.

5 Experiments

5.1 Data

We utilize four datasets for training and evaluation with dataset statistics provided in the appendix: the Stanford Sentiment Treebank (SST), comprising 11,855 movie reviews with 5 sentiment labels (ranging from 0 for Negative to 4 for Positive) 8; the CFIMDB dataset, containing 2,434 movie reviews for binary sentiment classification 7; the Quora dataset, consisting of 404,298 labeled question pairs for paraphrase detection 9; and a corpus of 154 Shakespearean sonnets for open-ended generation. We further add 1000 generated sonnets from the Gemini-2.5 family as discussed 10. For sonnet generation, we are limited by the fixed dev set of test set of 12 sonnets. We preface our results by acknowledging the limitations of our dataset and how an individual incorrectly-generated sonnet can greatly impact final performance.

5.2 Evaluation Method

For evaluation, we focus on task-specific metrics and baseline comparisons to assess model performance. For sentiment classification, we use accuracy as the primary metric, measuring the percentage of correctly classified sentences across sentiment classes in SST and CFIMDB datasets. For paraphrase detection, we use accuracy to measure the model’s ability to identify sentence pairs as paraphrases or non-paraphrases correctly in Quora dataset. For the sonnet corpus, we use the chrF metric [?], which is a character level n-gram comparison metric similar to BLEU.

5.3 Experimental Details

For our experiments on the three default project tasks, we utilized Modal and Google Cloud Platform with NVIDIA T4 and L4 GPUs. Our training configurations remained consistent across all experiments using the smallest GPT-2 model, with learning rates ranging from 1e-3 to 1e-5 and batch sizes of 4, 8, 16, 32 based on GPU memory constraints. We train for up to 20 epochs using dev set loss as validation loss to prevent overfitting. For open-ended sonnet generation, we implemented a new generate function. The model first applies temperature scaling ($T = 0.8$) to the output logits. To balance structural coherence with generation diversity, we use a hybrid approach that sequentially applies top- k truncation ($k = 40$) and top- p nucleus sampling ($p = 0.9$).

5.4 Results

Currently, our submission (LeBron) is first on the leaderboard for Sonnet Test with a score of 52.838 and 26th for Paraphrase Test with a score of 0.891.

5.4.1 Baseline Results

For all baseline experiments (sentiment classification, paraphrase detection, sonnet generation), we fully fine-tune GPT-2. We perform hyperparameter grid search on learning rate and batch size to obtain the highest performing models. For sentiment classification, we also provide results fine-tuning only the last layer of the network. All results are shown in Table 1. Downstream extension experiment best results are also shown, with test set performance only reported when present and limited due to 3 maximum submissions for a test score. For sentiment classification, full fine-tuned models had higher performance than their last linear layer fine-tuned counterparts which is expected as more task specific characteristics can be learned by the network as more parameters are allowed to be updated.

Table 1: Baseline performance across all tasks for validation (dev) and test sets. Sentiment classification and paraphrase detection use accuracy, and open-ended sonnet generation uses chrF score.

Task (Dataset)	Fine-Tuning Method	Metric	Dev Score	Test Score
Sentiment (SST)	Last Linear Layer	Accuracy	0.487	0.476
	Full Fine-Tuned	Accuracy	0.513	0.546
Sentiment (CFIMDB)	Last Linear Layer	Accuracy	0.865	—
	Full Fine-Tuned	Accuracy	0.971	—
Paraphrase (Quora)	Full Fine-Tuned	Accuracy	0.911	0.891
Sonnet Generation	Full Fine-Tuned	chrF	41.974	41.078
Sonnet Generation	Best LoRA	chrF	42.158	—
Sonnet Generation	Best Model Quantization	chrF	42.118	—
Sonnet Generation	Best Data Augmentation	chrF	46.605	52.838

5.4.2 LoRA

We explore LoRA for PEFT to determine if lower ranks can match performance of the fully fine-tuned baseline. By sweeping over learning rates from $1e-5$ to $5e-2$ with LoRA ranks 1 to 256, we find rank 256 with a learning rate of $1e-2$ is best performing with a dev chrF as 42.058. A batch size of 16 is used for these experiments due to memory constraints with the large scale of ablations run. All rank results are shown in Table 3 with all dev chrF scores being within 1 of the fully fine-tuned baseline. The table includes the number of trainable parameters with each rank to observe the computational efficiency achieved. For this task, even extreme low ranks can match performance of full fine-tuning. We hypothesize this is because GPT-2 has already learned the vast majority of English syntax, grammar, and vocabulary during pre-training. To adapt it to write sonnets, the model does not need to relearn English, but rather, a simple directional nudge towards sonnet structure.

We fixed α at 16, but with rank 256 and learning rate $1e-2$, we then ablate α (scaling factor) with 8, 16, 32, 64, 128, 256, 512 (up to $\alpha = 2r$) to successfully confirm the used $\alpha = 16$ was best [Table 5]. We see that LoRA requires a far higher learning rate than full fine-tuning, which is expected as the limited parameters being updated in LoRA can safely use a higher learning rate to ensure the updates are large enough to shift model predictions. Without a larger test set and bootstrapping we can't claim performance improvement, but we find that with an optimal learning rate and rank, LoRA can at least match a full fine-tuned baseline using fewer fine-tuning parameters for sonnet generation.

With the best rank of 256, learning rate of $1e-2$, $\alpha = 16$, we further investigate the impact of where LoRA is applied. We compare LoRA on all modules (attention, attention output, and MLP), versus attention and attention output (no MLP). Training all modules results in a higher dev chrF score of 42.158 compared to the no MLP training chrF score of 41.483 [Table 6] while maintaining a lower train and dev loss. We hypothesize that this is expected as the MLP contains roughly two-thirds of the dense parameters. By applying LoRA only to attention, there are simply not enough parameters being updated to capture the learned sonnet structure.

5.4.3 Model Quantization

In our exploration of quantization, we measure the efficiency we can gain in memory footprint and inference speed on sonnet generation. We measure the amount of space the model occupies in memory (MB) and compare this against the dev set chrF score achieved by these quantized models. As shown in Figure 3, switching to BF16 or FP8 halves the memory footprint of GPT-2 while offering minimal downside on inference. INT8 reduces the model to only 119 MB and INT4 reduces size to 60 MB, but performance drops by over 20% for this quantization. Since the performance drop is between INT8 and INT4, we test QAFT for these precisions, and performance is retained. chrF score only drops from 42.118 to 41.162 from our original FP32 GPT-2 model. Table 2 shows that lower precision formats, on top of providing considerable memory optimizations, also considerably optimize the number of tokens output per second during inference.

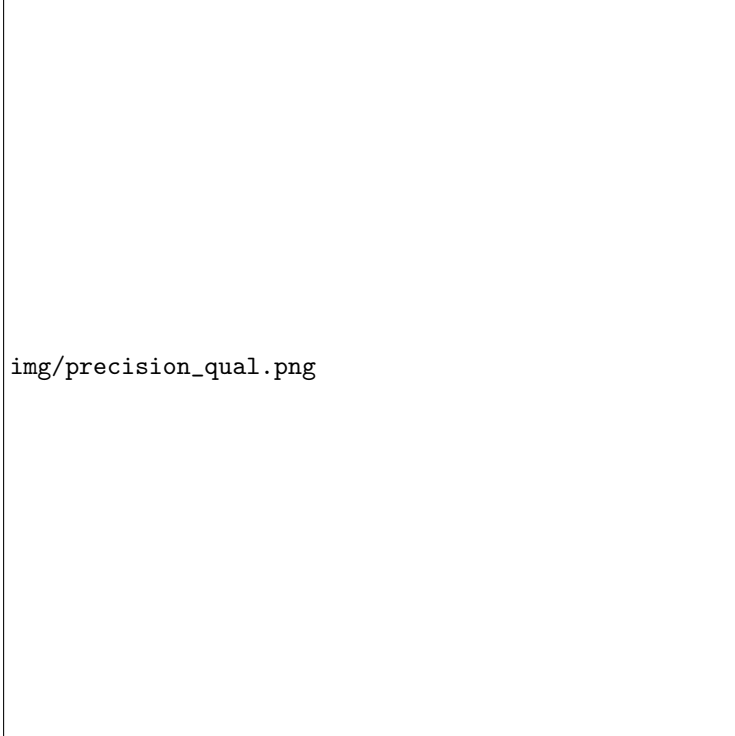


Figure 3: Sonnet generation performance relative to model size using quantization on GPT-2. Inference-only quantization applies post-training weight-only quantization to a fully fine-tuned sonnet model. QAFT applies weight quantization during SFT.

Table 2: Inference speed and model size comparison across different numerical precision formats for GPT-2 Small. Inference is performed on one L4 GPU with a batch size of 8.

Variant	Memory Footprint (MB)	Tokens/s
FP32	477.02	78.108
FP16	238.56	85.899
BF16	238.56	82.692
INT8	119.24	100.486
INT4	59.62	103.734

5.4.4 Synthetic Data Augmentation

We also explore performance improvement using synthetic data from a more complex model. The goal of this experiment is to not only see how synthetic data improves the performance of GPT-2, but also how using higher quality synthetic data across the Gemini 2.5 family affects performance.

In Figure 4 we see when using Gemini 2.5 Flash Lite, adding more data hurts performance since this model produces lower quality data. Flash Lite is optimized for speed and cost, not advanced reasoning or verbosity [?]. Flash is a model more optimized for complex tasks involving more advanced language and understanding (and 100× more expensive). When adding synthetic sonnet data from Flash, accuracy increases from our Baseline SFT performance of 42.243 to 46.60. Finally, we evaluate on Gemini 2.5 Flash Pro data. For SFT on subsets of 100 and 500 examples, its chrF performance is similar to that of 2.5 Flash. However, when fine tuning on all 1000 examples, it only reaches an accuracy of 43.236. It also costs ∼ \$6 to generate this data. We would assume that 2.5 Pro would produce the highest-quality in-distribution data and make our model overperform, but we found that its performance plateaus. This could be because Gemini 2.5 Pro is too large a teacher for such a small GPT-2 model, so distillation does not work. We further explore in the analysis.

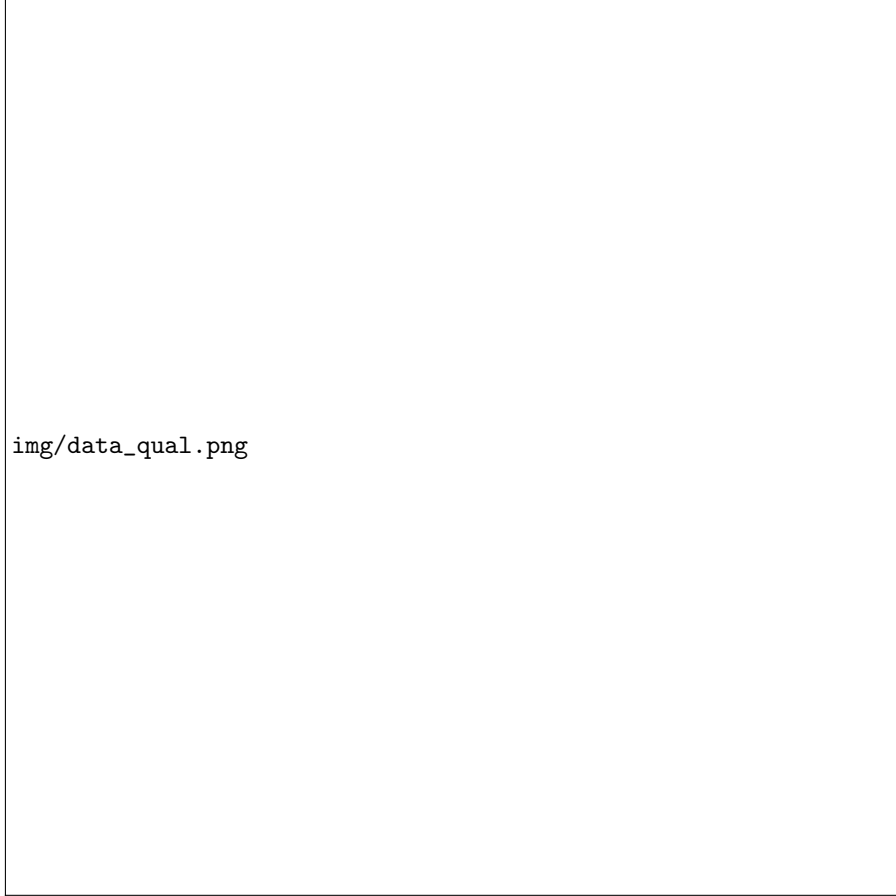


Figure 4: Comparison of the amount of synthetic data to dev CHRF. All subsets run on 20 epochs with a batch size of 32 and a learning rate of $1e-3$. Performing SFT on GPT-2 for sonnet generation.

6 Analysis

6.1 Baseline Experiment Analysis

As observed in the sentiment classification evaluations (SST and CFIMDB), full fine-tuning consistently outperforms linear probing (last layer fine-tuning). This performance gap indicates that the frozen pre-trained representations of GPT-2, while robust, are not perfectly linearly separable for these specific target domains. Furthermore, we find that hyperparameter sweeps are crucial to improving performance across tasks, and each task has different optimal hyperparameters. On small datasets like sonnet generation, we also found it important to include early stopping to prevent overfitting to the training data within early epochs.

6.2 LoRA

While LoRA results were found for the best learning rate at each rank, we further wanted to explore how the loss of LoRA compares to full fine-tuning sweeping over learning ranks for train and dev. We visualize this in Figure 5 with the justified $\alpha = 16$ with LoRA applied to all layers. Since early stopping was part of the training loop for overfitting, we show the train and dev loss at the last epoch. We see that the fully fine-tuned model reaches its lowest loss at a learning rate an order of magnitude smaller than the LoRA results. We hypothesize that LoRA optimizes a significantly lower-dimensional parameter space compared to full fine-tuning, which acts as an implicit regularizer and allows the model to tolerate larger optimization steps without destabilizing the pre-trained weights. We also see that the train loss for fully-fine tuned at $1e-3$ learning rate is significantly lower than any LoRA rank train loss, but the dev loss at the same learning rate is very high. This indicates the fully fine-tuned model is overfitting while most LoRA rank models maintain a consistent dev loss over the



img/lora_curves_train_dev_loss.png

Figure 5: Sonnet generation train and dev loss at the final epoch to account for overfitting across LoRA ranks and learning rates. A full fine-tuned baseline is included as a comparison.

sweep as it does not possess the ability to fully overfit since not all parameters can be updated. LoRA is more robust to overfitting while being more parameter efficient in the sonnet generation setting.

6.3 Model Quantization

Our quantized models matched relative performance of full precision in Figure 3. The INT4 format is a fraction of the size of FP32, yet retains 98% of chrF score with QAFT. The INT8 model also loses little performance. We hypothesize these QAFT models were overfitting to sonnet generation and losing performance on other tasks. In Figure 6, we perform QAFT on GPT-2 with INT8 precision for sonnet generation. Performance is relatively similar with QAFT compared to full fine-tuning. If we take an un-quantized GPT-2 model with SFT on sonnet generation and evaluate it on paraphrase detection, performance drops $\sim 28\%$. However, if we use the QAFT (INT8) from sonnet completion, zero-shot paraphrase detection performance drops $\sim 58\%$. QAFT has minor affects performance on a fine-tuned task, but it severely affects generalized performance, indicating overfitting.

Quantization reduces the numerical precision of model parameters, introducing approximation errors that can degrade downstream performance. Empirical studies show that low-bit quantization can impact zero-shot performance, as small changes in weights may disrupt the general representations learned during pre-training [? ?].

6.4 Synthetic Data Augmentation

For synthetic data, our hypothesis was that the model is dramatically overfitting to dev set sonnet generation, leading us to test its zero-shot abilities on paraphrase detection. According to the results in Table 7, even when fine-tuned on 1000 synthetic sonnets, the models do not lose zero-shot performance on dev set paraphrase detection, exhibiting an accuracy of 0.63. As shown before, the model fine-tuned on Gemini 2.5 Flash-Lite data does not perform better than baseline on the sonnet generation task. However, the model that uses Gemini 2.5 Flash data increases in chrF score by $\sim 10\%$ and loses no zero-shot accuracy. This demonstrates that the model fine-tuned on synthetic data is not overfitting to the sonnet generation task, and it is preserving some general ability to perform

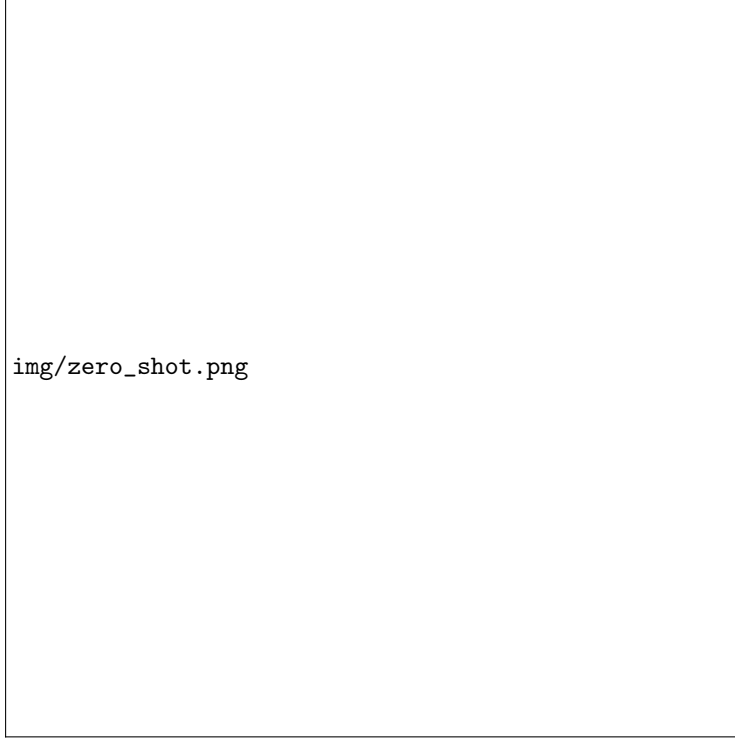


Figure 6: Overfitting of QAFT. GPT-2 Small is quantized to the INT8 format and then full fine tuned on sonnet generation. Its zero-shot performance for the paraphrase detection task is compared. All fine tuning run on 20 epochs with a batch size of 32 and a learning rate of $1e - 3$.

other (zero-shot) tasks. While neural networks are highly expressive and capable of memorizing training data, modern empirical results show that overparameterized models can still generalize well despite their capacity to overfit [?]. Our results suggest that synthetic distillation at this scale does not significantly degrade the model’s broader language capabilities.

For all three models, performance with 100 synthetic examples is slightly worse than the baseline [Figure 4]. The original training set contains only 143 human-written sonnets, so introducing 100 synthetic samples likely alters the training distribution. This can slightly degrade performance when only a small amount of synthetic data is added.

Even when fine-tuning on a frontier model’s data, a smaller student may struggle to fully distill knowledge from a much larger teacher [? ?]. In our experiments, using all 1000 Gemini 2.5 Pro-generated examples improves chrF performance on the sonnet generation task but does not surpass the performance achieved when distilling from Gemini 2.5 Flash. This non-monotonic behavior suggests that stronger teacher models do not necessarily produce more effective synthetic training data for smaller students. A possible explanation is a capacity mismatch: the complexity of Pro-generated sonnets may be hard for a GPT-2-scale model to imitate. We investigate if this finding exists when using GPT-2 Large. When fine-tuning the student GPT-2 Large model with synthetic Gemini Pro data, we find the distribution captured by the Gemini data still cannot be distilled properly. The chrF score on all 1000 examples is 43.716, indicating no real major improvement GPT-2 Small.

7 Conclusion

We evaluate empirical trade-offs across parameter, memory, and data efficiency when adapting GPT-2 under constrained compute and memory budgets. First, we demonstrate that LoRA can match full fine-tuned performance in sonnet generation while reducing trainable parameters and mitigating overfitting. Second, while inference-time quantization degrades performance at lower precisions, QAFT effectively stabilizes performance on sonnet generation. However, it risks task-specific overfitting and reduced zero-shot capabilities for paraphrase detection. Finally, our synthetic data

evaluation using the Gemini 2.5 family reveals that while higher-quality data improves distillation, GPT-2’s capacity limits prevent full knowledge transfer. Notably, augmenting training with synthetic sonnets preserves zero-shot paraphrase accuracy, suggesting this scale of augmentation does not induce catastrophic overfitting.

Despite these results, our study has several limitations. Our evaluations were constrained to GPT-2 (124M) architecture. While this provided a stable, efficient baseline for testing, it remains to be seen how these results and synthetic data scaling laws transfer to modern, massively scaled models. Additionally, while open-ended sonnet generation served as a stress test for structural adherence, it represents a narrow domain of text generation, motivating the need to investigate more complex tasks with larger datasets to further investigate overfitting and form statistically significant conclusions.

Team contributions

We all implemented GPT-2 and sentiment classification as the first part of the project together. Abhinav worked on hyperparameter sweeps and paraphrase detection. Ethan worked on synthetic data augmentation and quantization. Ryan worked on baseline sonnet generation and LoRA. We also all contributed to writing each respective part and creating the general flow of the paper.

References

- [1] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [2] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870. PMLR, 2018.
- [3] Ashvin Nair, Abhishek Gupta, Murtaza Dalal, and Sergey Levine. Awac: Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, 2020.
- [4] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.
- [5] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1889–1897. PMLR, 2015.
- [6] Dean A. Pomerleau. Alvin: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems*, volume 1. Morgan Kaufmann, 1989.
- [7] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning. *arXiv preprint arXiv:2110.06169*, 2021.

```

"""
Generate 1000 Shakespearean sonnets in English, in the style of Shakespeare's
original sonnets.

Hard constraints for EACH sonnet:
- Exactly 14 non-empty lines.
- No title line.
- Strict end-rhyme scheme: ABAB CDCD EFEF GG (clear end rhymes; avoid slant/near
rhymes).
- Shakespearean diction and syntax (Early Modern English feel; no modern slang).

Formatting constraints:
- Before each sonnet, output a single line containing exactly: ###{
starting_index}### for the first sonnet, then increment by 1 for each next
sonnet.
- After the delimiter, output the 14 lines of the sonnet.
- Output nothing else.

Begin.
"""

```

Listing 1: The prompt template used for generating Shakespearean sonnets from Gemini-2.5 models.

Table 3: LoRA rank and validation chrF score for sonnet generation. For each rank, we report the optimal learning rate that yielded the highest dev chrF score using a constant batch size of 16 and $\alpha = 16$ as well as the trainable parameters in each setting. Learning rate was swept over between $1e-5$ and $5e-2$. A full fine-tuning baseline is included.

Configuration	Trainable Parameters	Optimal Learning Rate	Dev chrF Score
LoRA Rank 1	165,888	5×10^{-3}	41.679
LoRA Rank 2	331,776	1×10^{-3}	41.763
LoRA Rank 4	663,552	3×10^{-5}	41.369
LoRA Rank 8	1,327,104	1×10^{-3}	41.927
LoRA Rank 16	2,654,208	5×10^{-4}	41.777
LoRA Rank 32	5,308,416	1×10^{-3}	42.008
LoRA Rank 64	10,616,832	5×10^{-4}	41.876
LoRA Rank 128	21,233,664	5×10^{-3}	42.020
LoRA Rank 256	42,467,328	1×10^{-2}	42.158
Full Fine-Tuning	$\sim 124M$	7×10^{-5}	41.9093

Table 4: Percentage of generated sonnets that match the Shakespearean rhyme scheme (ABAB CDCD EFEF GG) using CMU Pronouncing Dictionary rhyme detection and are 14 lines.

Model	Perfect Sonnet Rate (%)
Gemini 2.5 Flash Lite	56%
Gemini 2.5 Flash	72%
Gemini 2.5 Pro	75%



img/ids_cfimdb_stats.png

Figure 7: Dataset statistics for CFIMDB used for sentiment classification. A train, dev, and test split was used with labels shown for train and dev label distributions.

Table 5: Impact of the LoRA scaling factor (α) on dev chrF performance for sonnet generation. Hyperparameters were based on previous ablations with batch size 16, learning rate 1e-2, rank 256.

α Scaling Factor	8	16	32	64	128	256	512
Dev chrF	41.824	42.158	41.777	40.041	35.531	36.087	37.057

Table 6: Ablation study of targeted LoRA modules. Restricting LoRA to the attention mechanism and freezing the Multi-Layer Perceptron (MLP) results in a measurable drop in validation performance. All layers represents the attention, attention output, and MLP layers.

Modules Trained	Train Loss	Dev Loss	Dev chrF Score
All (Attention + MLP)	2.404	4.170	42.158
No MLP (Attention Only)	3.457	4.355	41.483

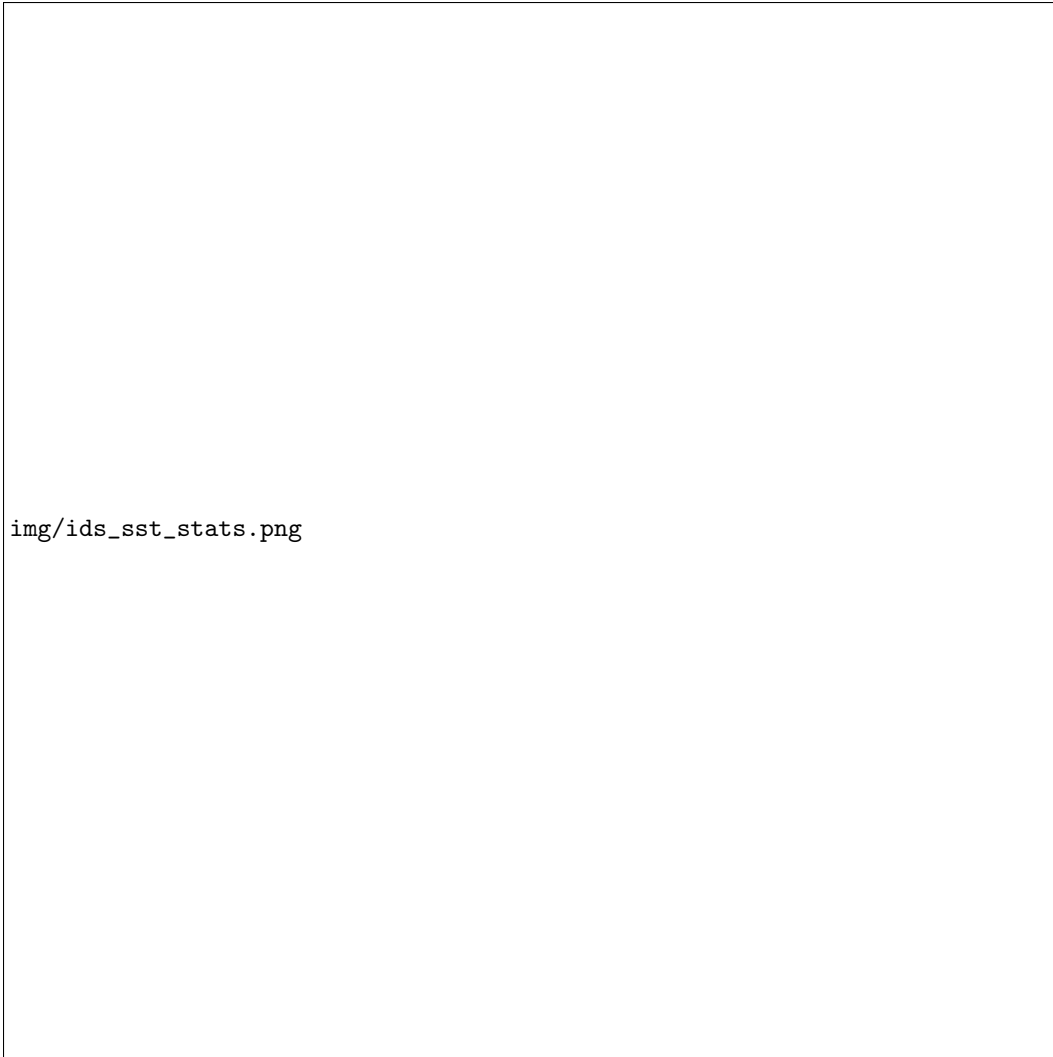


Figure 8: Dataset statistics for SST used for sentiment classification. A train, dev, and test split was used with labels shown for train and dev label distributions.

Table 7: Zero shot capabilities of SFT sonnet generation with a baseline model fine-tuned on provided sonnet data and models fine-tuned with additional synthetic data evaluated on paraphrase detection. All models exhibit similar paraphrase accuracy despite 2.5 Flash SFT’s superior sonnet generation.

Model	Sonnet Generation Dev chrF	Paraphrase Dev Zero Shot Accuracy
Sonnet Generation SFT (Baseline)	42.243	0.6317
2.5 Flash SFT (1k)	46.600	0.6317
2.5 Flash-Lite SFT (1k)	42.092	0.6315



Figure 9: Dataset statistics for Quora used for paraphrase detection. A train, dev, and test split was used with labels shown for train and dev label distributions.

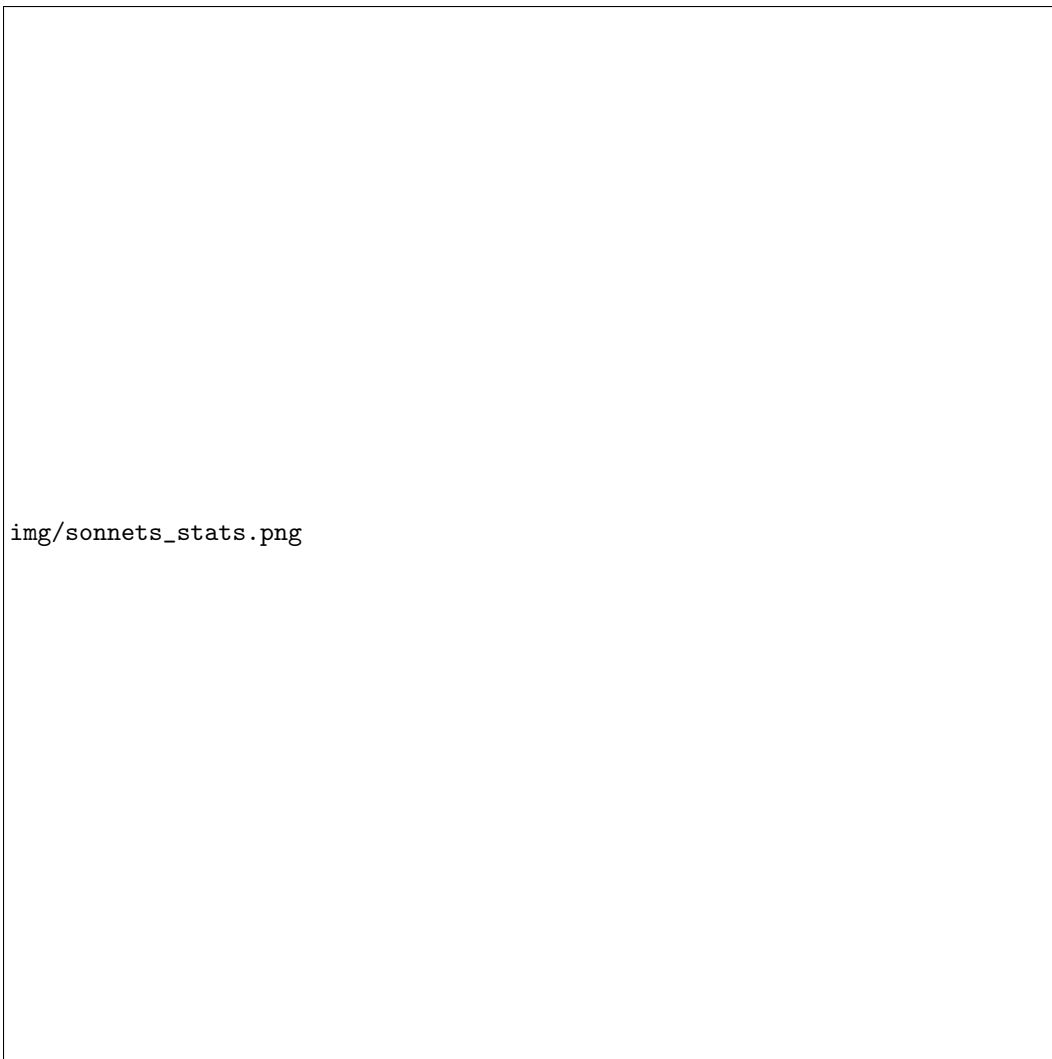


Figure 10: Dataset statistics for sonnets used for paraphrase detection. A train, dev, and test split was used with synthetic data added in for training.

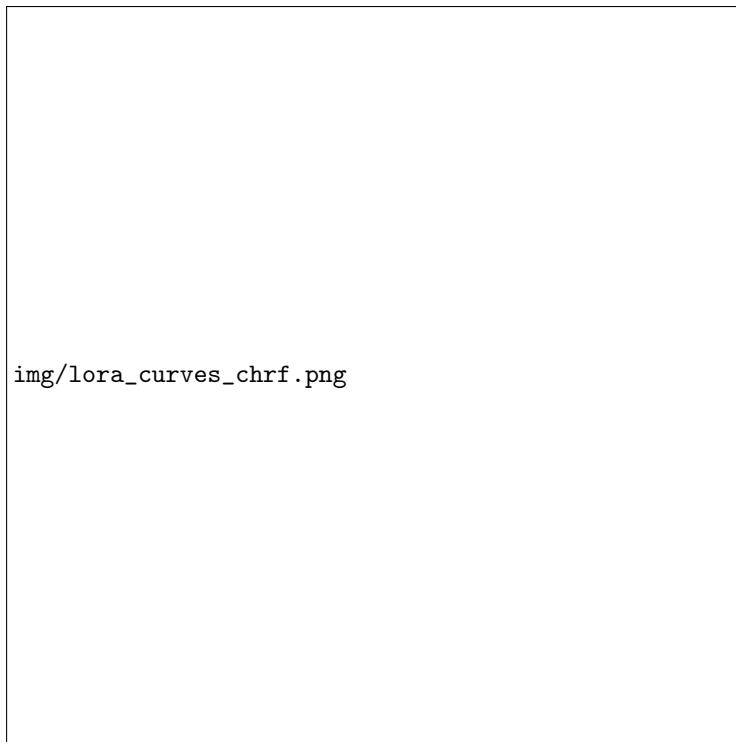


Figure 11: Sonnet generation dev chrF score for LoRA ranks across learning rates. A full fine-tuned baseline is included. Rank 256 has the best dev chrF performance of 42.058 at a learning rate of 1e-2